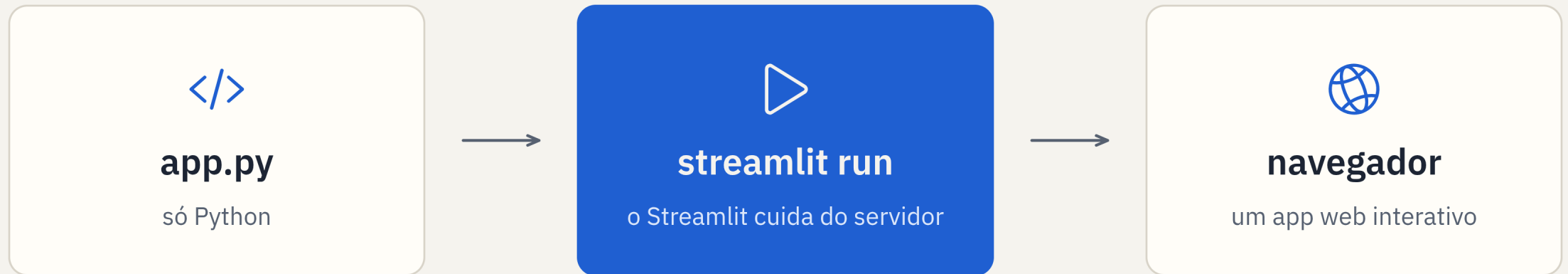




Introdução ao Streamlit

Um app web escrito só em Python

O que é o Streamlit



sem HTML

sem JavaScript

sem configurar servidor

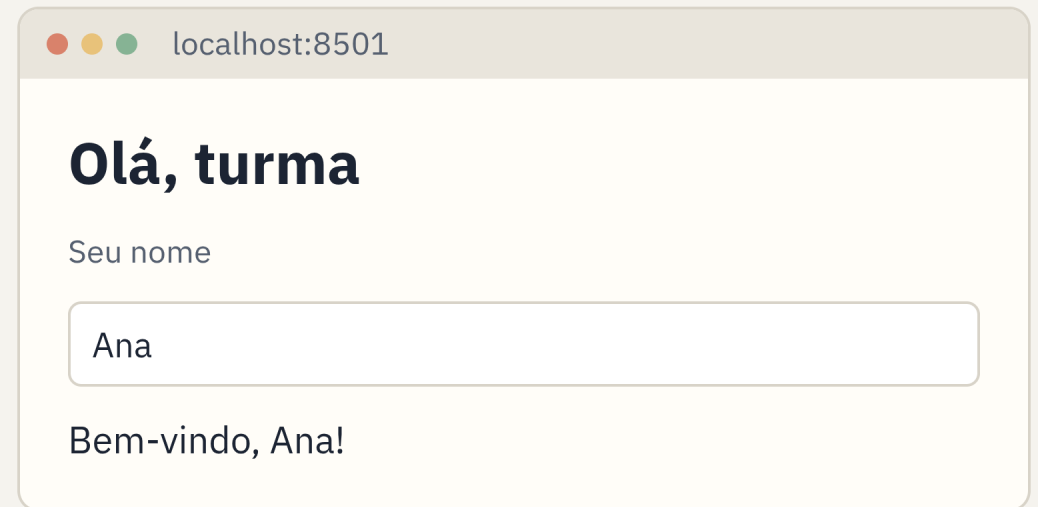
O primeiro app: quatro linhas

app.py

```
import streamlit as st

st.title("Olá, turma")
nome = st.text_input("Seu nome")
st.write(f"Bem-vindo, {nome}!")
```

```
$ streamlit run app.py
```



Mostrar dados: tabela, gráfico e métrica

```
st.metric("Último preço",  
          "R$ 38,45", "+0,9%")  
  
st.line_chart(df, x="data",  
              y="fechamento")  
  
st.dataframe(df)
```



Widgets: o usuário interage

Gerar carteira

```
st.button("Gerar carteira")
```

Setor

Bancos ▼

```
st.selectbox("Setor", setores)
```

Número de ações: 5



```
st.slider("Número", 1, 20, 5)
```

Nome

Minha carteira

```
st.text_input("Nome")
```

Quantidade

5 – +

```
st.number_input("Qtd", 1, 20)
```

Foto

Arraste um arquivo

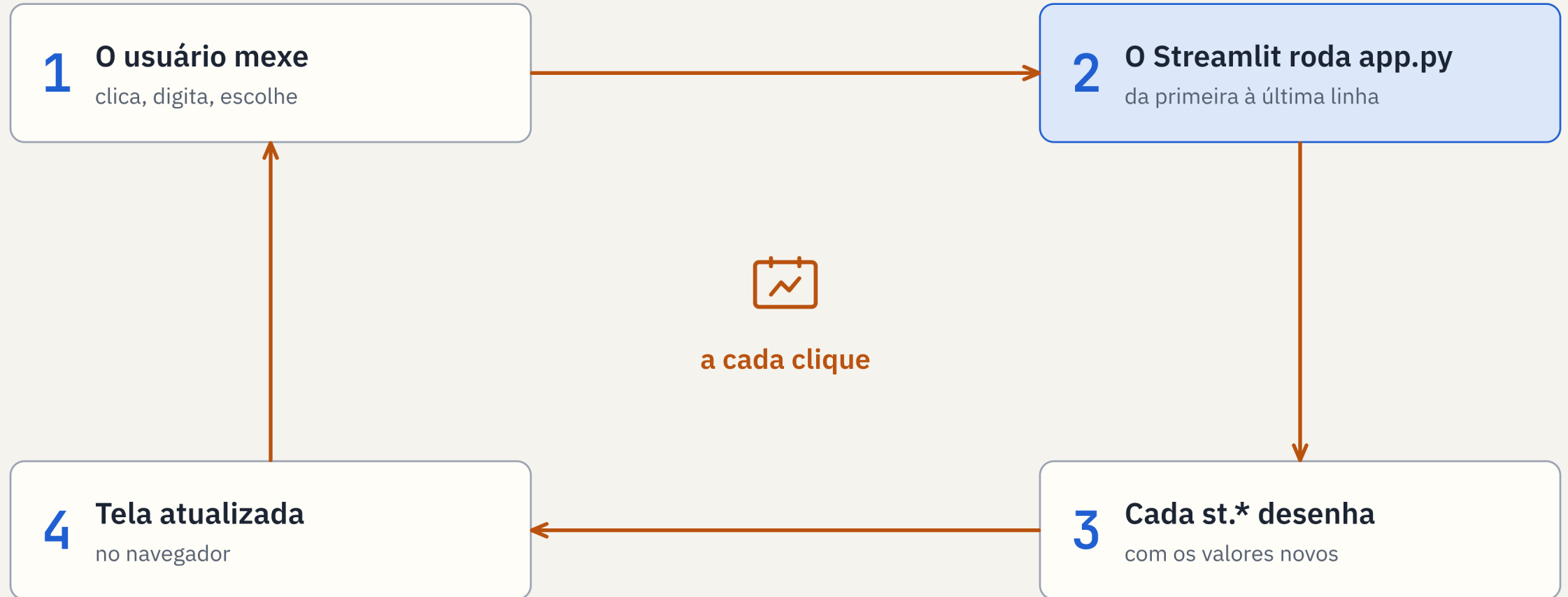
```
st.file_uploader("Foto")
```

01

O segredo: o script roda de novo

reexecução · session_state · forms · cache

A cada interação, o script roda do início ao fim



O problema: variáveis comuns esquecem

```
contador = 0

if st.button("+1"):
    contador += 1

st.write(contador)
```

1º clique

1

2º clique

1

3º clique

1

A cada clique o script recomeça, e a primeira linha volta o contador para 0

st.session_state: a memória de cada usuário

```
if "contador" not in st.session_state:  
    st.session_state.contador = 0  
  
if st.button("+1"):  
    st.session_state.contador += 1  
  
st.write(st.session_state.contador)
```

1º clique

1

2º clique

2

3º clique

3

aba da Ana

contador = 3

aba do Bruno

contador = 1

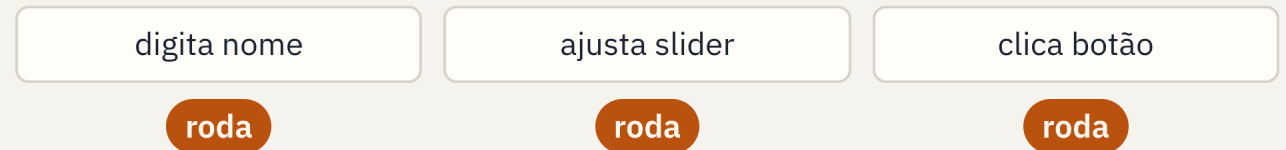
cada aba do navegador tem o seu

st.form: um envio, uma execução

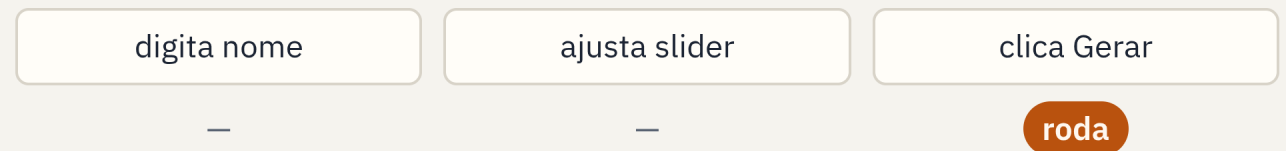
```
with st.form("carteira"):
    nome = st.text_input("Nome")
    n = st.slider("Ações", 1, 20, 5)
    ok = st.form_submit_button(
        "Gerar")

if ok:
    montar(nome, n)
```

Sem form



Com form



Cache: não refazer o trabalho caro

```
@st.cache_data(ttl=600)
def carregar_precos(ticker):
    # consulta lenta ao banco
    ...
    return df

df = carregar_precos("PETR4")
```



1ª chamada com PETR4

executa a função e guarda o resultado



chamadas seguintes com PETR4

devolve o resultado guardado, sem consultar

ttl=600: o resultado vale por 10 minutos

Dois caches, dois propósitos



`st.cache_data`

guarda resultados: tabelas, listas

cada chamada recebe uma cópia

ex.: preços consultados no banco



`st.cache_resource`

guarda objetos: modelo, conexão

um único objeto para todos os usuários

ex.: um modelo de ML carregado uma vez



O login de um usuário nunca vai para o `cache_resource`

ele é compartilhado: o próximo visitante herdaria a sessão do anterior. Login fica em `st.session_state`.

02

Do script ao app publicado

várias páginas · segredos · publicação · conexão com o banco

Várias páginas: st.navigation

```
app/  
├─ streamlit_app.py  
└─ paginas/  
    ├─ ranking.py  
    ├─ acao.py  
    ├─ carteiras.py  
    └─ perfil.py
```

```
pg = st.navigation([  
    st.Page("paginas/ranking.py",  
            title="Ranking"),  
    st.Page("paginas/perfil.py",  
            title="Meu perfil"),  
])  
pg.run()
```

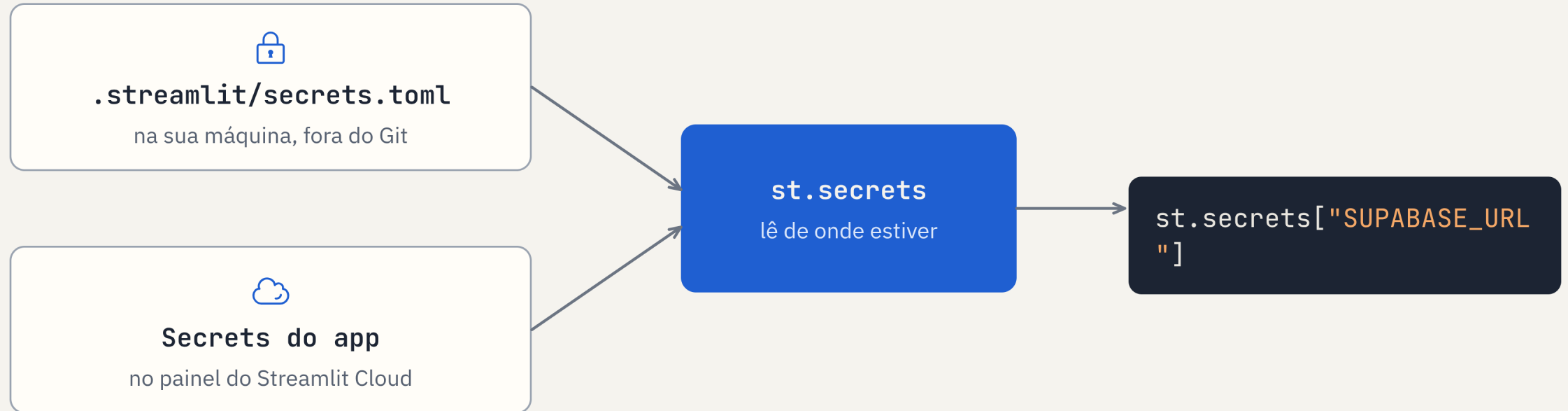
Ranking

Ação

Minhas carteiras

Meu perfil

Segredos: st.secrets



Publicar de graça: Streamlit Community Cloud



git push → o app se atualiza sozinho

12 h sem acesso → hiberna, ~1 min para acordar

Ponte com a Aula 1: ler o Supabase

```
from supabase import create_client

sb = create_client(st.secrets["SUPABASE_URL"],
  st.secrets["SUPABASE_PUBLISHABLE_KEY"])
sb.auth.sign_in_with_password({
  "email": st.secrets["EMAIL"],
  "password": st.secrets["SENHA"]})

dados = sb.table("ativos")
  .select("ticker, nome").execute().data
st.dataframe(dados)
```

localhost:8501

Ações

ticker	nome
ABEV3	Ambev
B3SA3	B3
BBAS3	Banco do Brasil
BBDC4	Bradesco PN

Sem login, a RLS não libera nada: o exemplo entra com um usuário de teste

Recapitulando



st.* desenha

na ordem do script



Dados

tabela, gráfico, métrica



Widgets

devolvem valores



Reexecução

a cada interação



session_state

memória de cada aba



st.form

um envio, uma execução



Cache

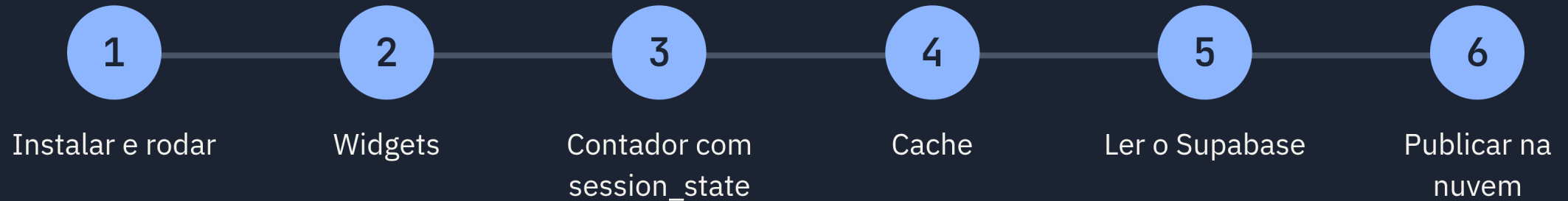
data por cópia, resource
para todos



Publicar

páginas, secrets, Cloud

Agora, ao vivo



```
pip install streamlit supabase · streamlit run app.py
```